CURS - 4 - 1.

(1) — Transferul ~~de~~ între registre:

- transferul conținutului unui registru sursă într-un registru destinație, fără a afecta conținutul sursei, se notează astfel: AC = RD

Dacă registrul AC are patru biți, numărul / factorul în unul a tuturor biștabililor se poate nota după cum urmează: AC = 4'b'0000, AC = 4'b'1111,

(2) — semnale de tip nivel (SCN)

↳ un circuit secvențial de comandă furnizează, pentru rețeaua de date, diverse semnale de comandă, sincrone cu ceasul sistemului, cu perioade egale de durată unei perioade de ceas sau multiple ai acestuia.

(2) — semnale de comandă de tip impuls (SCI)

↳ semnalele de comandă de tip nivel (SCN) vor avea un sufix numeric (0 ... 9) ce specifică numărul ieșirii de comandă a automatului (SCNi). Semnalele SCNi pot fi sincronizate cu semnalul curent de ceas.

(3) — Sistem sincron & asincron

→ Pentru toate sistemele numerice este necesară o convenție în legătură cu momentul în care receptorul / destinația poate prelua date aplicate la intrarea sa:

- sistemele sincrone utilizează un semnal de ceas comun
- sistemele asincrone codifică semnalele "date prezente" sporit nou împreună cu semnalele care reprezintă datele.

→ Sistemele numerice, necesită o convenție privitoare la momentul în care emițătorul poate transmite în siguranță o altă dată / informație către receptor:

- sistemele sincrone: la următorul front al sistemului de ceas
- sistemele asincrone: după emiterea de către receptor a semnalului recepționat

(1) — Implementarea registrelor se bazează pe biștabile JK sau D, cu intrări de sincronizare de ceas CLK și cu intrări de tip $\overline{PRESET}$ / $\overline{CLR}$

- Transformările sincrone au loc sub controlul semnalului de ceas:

D → Pozitiv & JK → Negativ.

Scanned with CamScanner

(4) Întârzierea în circ combinațional.

Constă prin elemente de tip CMOS - cele 2 tensiuni, cu intrare-ieșire, olor nu instantaneu.

Întârzierea totală de propag = Σ int inter.

Drum critic = calea cea mai lungă dintre N căi posibile.

(6). SKEW: variația spațială a timpilor de sosire a semn ck cu variația în care se privește a celor front de ceas, văzut de 2 sau mai multe bistabile diferite.

JITTER: variația temp. a timpilor de sosire: variație în care se privește timpii de sosire a 2 fronturi succesive ale semn ck ceas la același bistabil.

Cauza: zgomotul surse de alimentare.

(8) Cele mai mari dificultăți se regăsesc la tehnicile folosite pentru realizarea unor interconexiuni cât mai flexibile în cadrul blocurilor și între blocuri.



Conexiune de tip „punct de intersecție" în cadrul unor trasee de tip „magistrală intersectată" sau cross-bar

La intersecția între barele verticale și cele orizontale se pot stabili conexiuni permanente sau temporare, în funcție de tehnologia utilizată.

- Permanente → se utilizează elemente de tip „anti-fuse" cu contact permanent stabilit ca urmare a aplicării temporale a unei tensiuni ridicate.

Avantaje: caracter nevolatil, dimensiuni relativ mici, rezistență și capacitate reduse.

Dezavantaje: conținutul fix, imposibilitatea reprogramării

- Temporare: comutatoare tranzistoare NMOS, cu canal indus sau tranzistoare cu poartă flotantă.

Avantaje: posibilitatea reconfigurării.

Dezavantaje: caracter nevolatil, dimensiuni relativ mari ale comutatoarelor.

(6) Restricții pt drumul cel mai lung sau cel mai lent:

$T_{ceas} \geq T_{cq\,max} + T_{p\,max} + T_{stabilă/setup} + T_{olumicot/sheru}$

Restricții pentru drumul cel mai scurt / calea cea mai rapidă

$T_{cq\,min} + T_{p\,min} \geq T_{menținere/hold} + T_{olumicot/sheru}$.

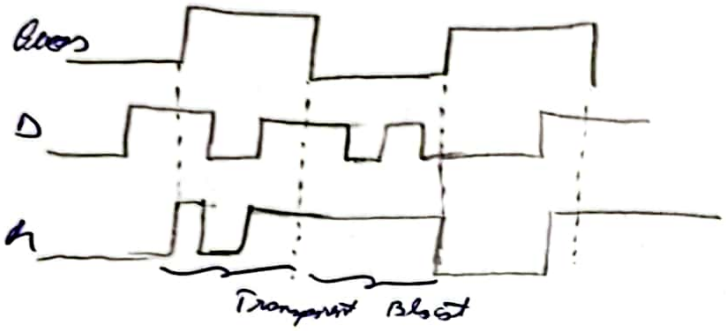
Scanned with CamScanner

SUB CURS 4

5) • Diferența dintre un latch transparent și un bistabil D comandat pe front.

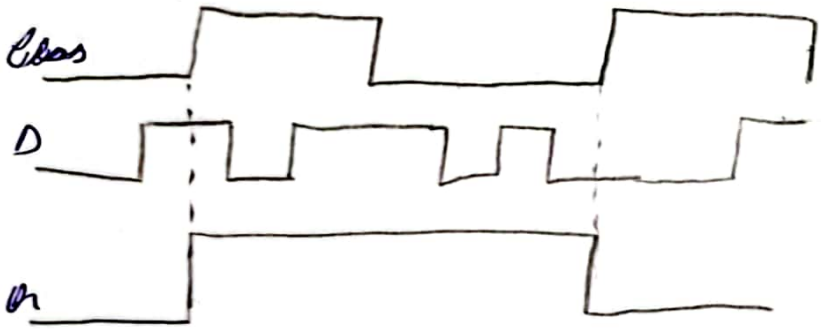
Latch transparent

Când semnalul de ceas este la nivel înalt, datele traversează latch-ul, iar când semnalul de ceas este la nivel coborât, datele sunt blocate.



Bistabil de tip D

Datele sunt stocate/capturate pe frontul crescător al circuitului și memorate pe restul circuitului.



• Parametrii sincronizării / timing-ului latch-ului:

→ $T_{CQ\,min}$ / $T_{CQ\,max}$ → timpul de propagare a semnalului de la intrare la ieșire atunci când semnalul de ceas deschide latch-ul

→ $T_{DQ\,min}$ / $T_{DQ\,max}$ → timpul de propagare a semnalului de la intrare la ieșire atunci când latch-ul este transparent. (cel mai important parametru de sincronizare a ceasului)

→ $T_{stabilire}$ / setup / $T_{menținere}$ / hold → definesc o fereastră în jurul frontului pozitiv al semnalului de ceas pe durata căreia data trebuie să fie stabilită pentru a putea fi eșantionat corect

Scanned with CamScanner

SUB CURS 4

• Curs de referire la timpul de stabilire și timpul de menținere în cazul latch-urilor.

Stabilire / Setup - reprezintă acea privind propagarea noii date pe bucla de reacție înainte ca ceasul să închidă poarta de la intrare.

Timpul de reținere reprezintă cursa pentru semnalul de ceas privitoare la închiderea porții de intrare ca următorul ciclu de date de la intrare să perturbe valoarea stocată.

CURS-6.

(7) • FPGA → unitățile structurate

Componentele structurate de bază ale arăților de porți ale tip FPGA sunt:

- blocuri logice programabile
- comutatoare programabile
- trasee de interconectare (routing)

Blocurile logice pot realiza sub formă de:

→ rețele de perechi de tranzistoare NMOS și PMOS, comutatoare de tip T-gates.

→ rețele de porți combinaționale (NAND, XOR, etc.)

→ multiplexoare

→ tabele asociative (lookup tables) - cu n intrări

→ structuri ȘI-SAU cu mai multe intrări.

Trasee de interconectare → Xilinx
→ Actel
→ Altera
} producători

• FPGA → conexiune cross-point.

Familiile de FPGA-uri diferă prin → mijlocul fizic de realizare a programării de către utilizator.

→ organizarea traseelor de interconectare

→ funcțiile de bază ale blocurilor logice combinaționale (CLB)

Scanned with CamScanner

conexiuni directe V_S prin magistrale.

- directe: — oferă grad mare de paralelism al transf de date
  ↳ consumă multe pini pentru circuit - ex porți logice.
- magistrale - se reduce costul conexiunilor între surse și destinații
  - transferurile nu se vor mai efectua paralel, vor avea caracter secvențial.

Curs 5

- Arhitectura calculatorului numeric:

  $A = \langle P_i, PE, RG, I \rangle$

  P_i → mulțimea porturilor de intrare.

  PE → — " — porturile de ieșire.

  RG → — " — reg. generale din unitatea de execuție

  I → setul instrucțiunilor.

- Reprezentarea generală a unui sistem de calcul:

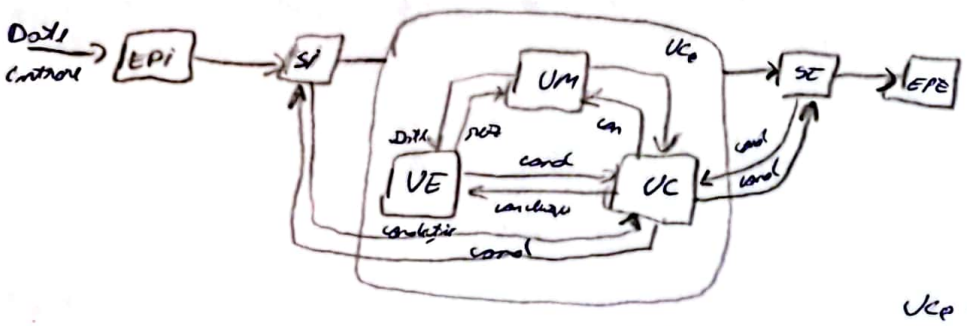
  structura generală → unitatea de intrare (UI)
  → unitatea centrală (UC)
  → unitatea de ieșire (UE)

  U_i, UE → legătura cu echipamentele periferice primare, cu care face schimb de date, info.

  U_i → S_i (subsistem de intrare); EPI (echipamente periferice de intrare)

  UCe → asigură starea inițială, a programului și preluarea datelor de către programul.

  UCd → unitatea de memorie (UM), execuție (UE) și comandă (UC)



$C = \overset{\text{num}}{M} + P + \overset{\text{terminal}}{T}$

P → prac

$UE + UC = P$

$P + M = UCe$

$UCe + SI + SE + Soft = Sist\ de\ calcul$

Scanned with CamScanner

UM → stocare obiecte, programului, rez. intrări și ieșiri

UE → operații aritmetice și logice.

UC → prelucrează fluxul de instrucțiuni și dă semnale de comandă

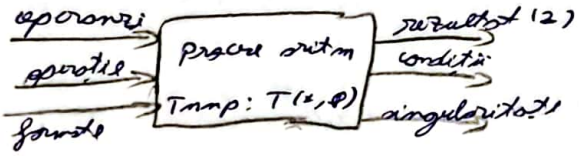
→ instrucțiuni: → operaționale și de transfer, inclusiv de I/E
└→ decizionale.

Câmpuri → codul operației / funcția
└→ adresa operandului / instrucțiunii

Exemple: ~~Q1 și MIPS (toate de 32 biți)~~
R → instrucțiuni aritmetice

Curs 6-1. Procesor aritmetic.

→ Un procesor aritmetic este un dispozitiv capabil să efectueze operații asupra unor operanzi furnizați în formate cerute.



Proiectare aritm → Proiectare logică (tipurile de arc logică).

Sumatoare:

→ registru acumulator al rezultatului AC

- Indicatorii de condiții: bistabile notate cu mnemonic:
rezultat - Z, semnul rez - S, carry - C, paritate - P

Sumator paralel:

→ cu transport succesiv: 2 funcții combinaționale pe 2 nivele logice

$$t_{out_i} = (x_i \& c_{in_i})(y_i \& c_{in_i}) \mid (x_i \& y_i)$$

$$sum_i = (x_i \& \bar{y}_i \& \bar{c}_{in_i})(\bar{y}_i \& x_i \& \bar{c}_{in_i}) \mid (\bar{x}_i \& \bar{y}_i \& c_{in_i}) \mid (x_i \& y_i \& c_{in_i})$$

→ cu întârziere minimă:

$$s_0 = a_0 \uparrow b_0 \uparrow c_0$$

$$s_1 = a_1 \uparrow b_1 \uparrow \underbrace{(a_0 \& b_0)\, a_0 \& c_0 \mid b_0 \& c_0)}_{c_1}$$

→ sumator carry - look - ahead.

$$c_{in\,1} = b_0 \cdot a_{in_0} + a_0 \cdot c_{in_0} + a_0 b_0$$

$$c_{i+1} = a_i \cdot b_i + c_i \cdot (a_i + b_i) = g_i + p_i \cdot c_i$$

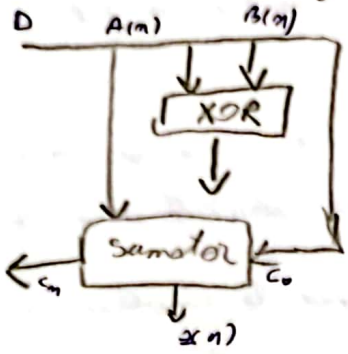
↳ propag. transport

gen. transport

$$sum_i = p_i \uparrow c_i$$

Scanned with CamScanner

Operațiile de adunare și scădere ale numerelor în virgulă fixă se implementează, în majoritatea cazurilor, cu numere reprezentate în complementul față de doi.



op	Descriere	Comanda D
ADD	$Z = C_4, ADD(A,B)$	0
SUB	$Z = C_4, SUB(A,B)$	1

1) Sumatoare cu transport succesiv
- tipul cel mai simplu de sumator paralel, obținut prin interconectarea unor sumatoare complete
- cel 2 fel combinațional se implementează pe 2 nivele (întârziere min $= 2\Delta t$)

2) Sumatoare cu întârziere minimă
- Considerând ec. sumei sume la ieșirea celui mai puțin semnificativ rang, se încearcă stabilirea unei expresii a acestei, de fapt depășind atât de intrare pt rangul curent $(n-1)$, cât și pt rangurile mai puțin semnificative $(n-2) \dots 1, 0 \Rightarrow$ expresia logică implementabilă în 2 trepte.

3). Sumă cu anticiparea transportului (carry look ahead).

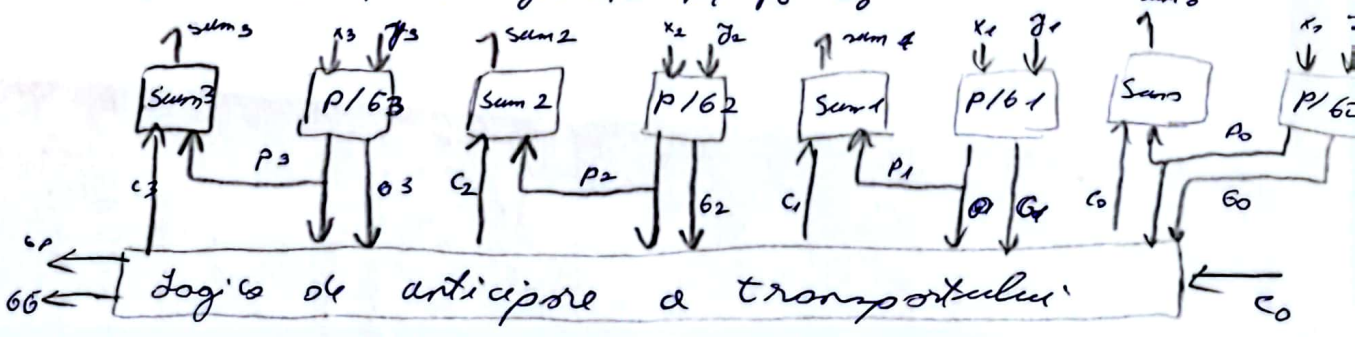
$c_{in1} = b_0 c_{in0} + a_0 c_{in0} + a_0 b_0$

$C_{i+1} = b_i \cdot c_i + a_i \cdot c_i + a_i \cdot b_i = \underbrace{a_i \cdot b_i}_{\text{generează transport } g_i} + c_i (a_i + b_i) \Rightarrow C_{i+1} = g_i + p_i \cdot c_i$ (propagare transport p_i)

$c_1 = g_0 + p_0 \cdot c_0$

$c_2 = g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 + c_0$

$c_3 = g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 = g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0$



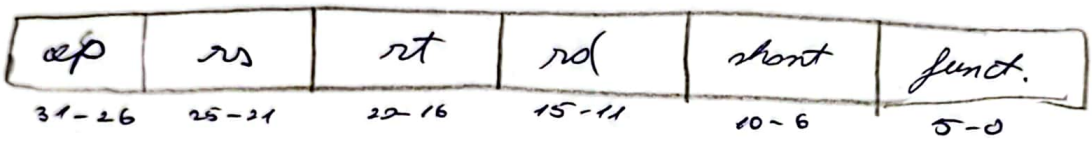
Scanned with CamScanner

MIPS = microprocesor without interlocking Pipeline Stages

Subiect → Explicați execuția instrucțiunii de tip R pentru un procesor MIPS care rulează pe un singur ciclu de ceas, punând în evidență formatul instrucțiunii unitățile funcționale implicate, registrii cheie.

- Instrucțiuni de tip R.
  - tipul R → instrucțiuni aritmetico - logice
    ↳ add, sub, and, or, slt (set-on-less-than)

- Formatul instrucțiunii



op - codul de operație al instrucțiunii.

rs, rt, rd = adresele registrelor sursă și destinație.

shamt = numărul de biți cu care se efectuează deplasarea.

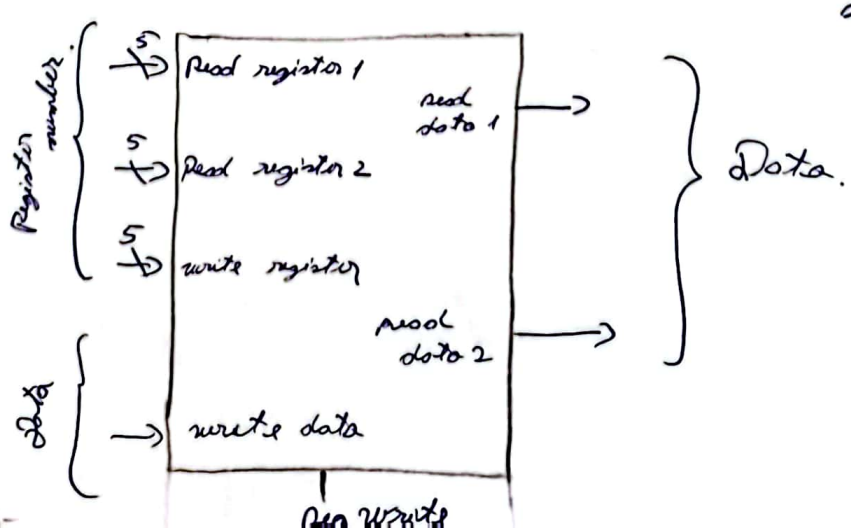
funct = selectează varianta de operație specifică de către op.

- ~~op~~. Rezultatul din UAL trebuie scris într-un registru.
  - → aritmetice: sub, add
  - → logice: and, or, xor, slt (set-on-less-than)

Instrucțiunile de format R au ca operanzi 3 registre. → 2 sunt citite, unul este scris.

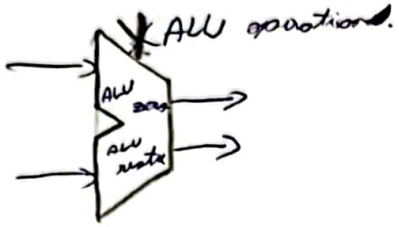
- Pentru realizarea căii de date sunt nevoie de:

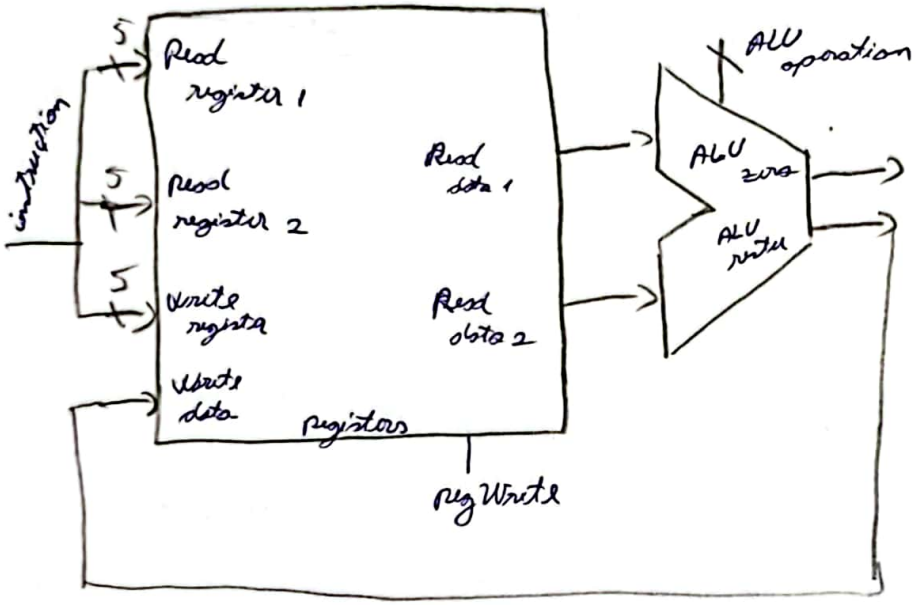
a) Registrele generale



Scanned with CamScanner

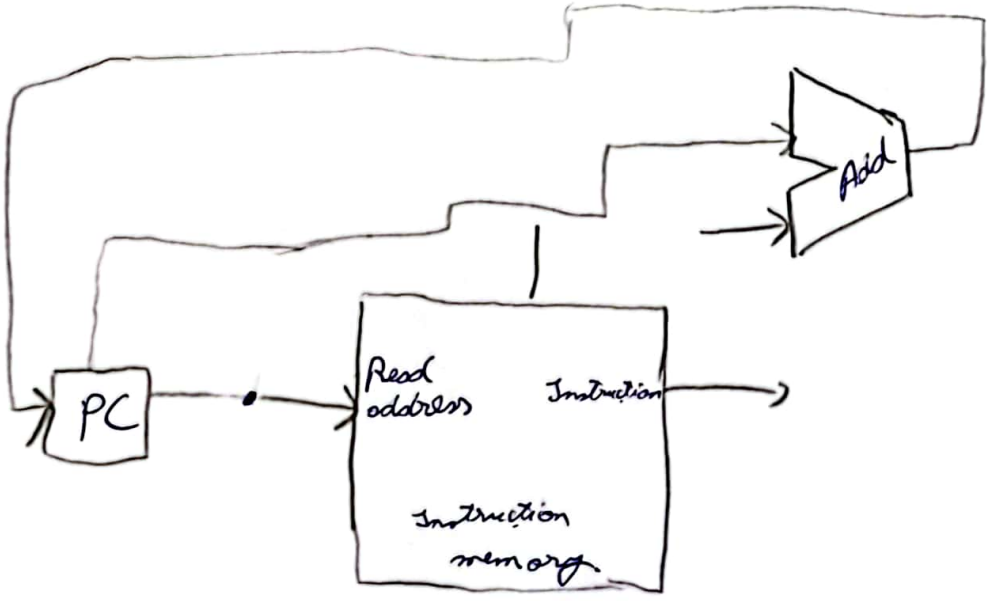
h) UAL
unitate aritmetico-logică
↓
operează cu valori aflate din regiștri





Obținerea instrucțiunii curente și trecerea la următoarea instrucțiune MIPS se un singur ciclu de ceas.

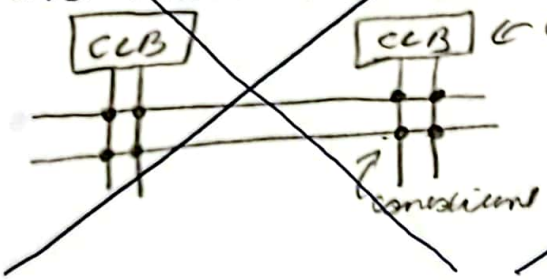
→ se extrage instrucțiunea de memorie
→ pentru trecerea la următoarea instrucțiune, aflată la 4 octeți distanță se incrementează PC-ul cu 4, acesta este element de stare.



Scanned with CamScanner

CURS 4 (lucrul cu registrele)

(5) Cele mai noi dispozitive ce regăsesc la tehnicile folosite pentru realizarea unei interconexiuni sunt mai flexibile în cadrul blocurilor și între blocuri



CLB ← configurabil

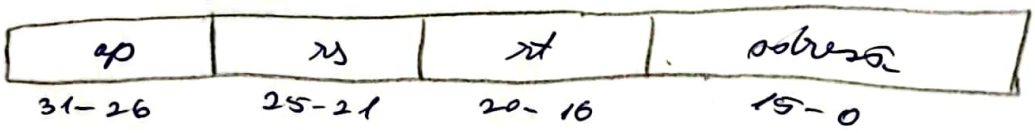
conexiuni de tip „punte de întrerupție" în cadrul unor trasee de tip „registrele întrerupte" sau „pass-bar".

SUBIECT: Instrucțiunea de ramificație procesor MIPS - un singur ciclu de ceas

→ formatul instrucțiunii
→ unitățile funcționale implicate.
→ registre check.

→ instrucțiune de tip I. (de transfer, ramificație, imediat)

Format:



op → codul de operație al instrucțiunii

rs → adresa registrelor sursă (source)

rt → adresa registrului sursă (target)

adresa → deplasarea adresei (offset) / valoarea imediată

Se folosește ieșirea ALU în determinarea adresei următoarei instrucțiuni de executat → se introduce o logică de control

Ramificațiile → egal
↓ nu
↓ mai

Instrucțiunea beq (ramificație întârziată)

Instrucțiunea este formată din 2 registre, care sunt comparate pentru a egalitate și o deplasare de 16 biți, folosită în calculul adresei obiectiv pentru ramificație:

beq: $t1, $t2, offset

În vederea implementării acestei instrucțiuni trebuie determinată adresa obiectiv pentru ramificație → PC+ conținul de deplasare al instrucțiunii cu semn extins.

Offset este în calculul adresei obiectiv pentru ramificație care este relativă la adresa instrucțiunii de ramificație.

Conținul deplasării trebuie mutat cu 2 poziții la stânga.

Scanned with CamScanner

SUBIECT Explicați execuția instrucțiunilor de încărcare/memorare pentru un procesor MIPS care rulează pe un singur ciclu de ceas, punând în evidență formatul instrucțiunii, unitățile funcționale implicate, registrul cheie.

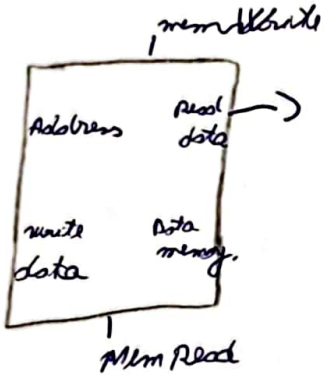
INSTRUCȚIUNILE DE TIP I → instrucțiuni de referire a memoriei

Bitrâm de pe poz precedentă în operand [ce a încărcat]

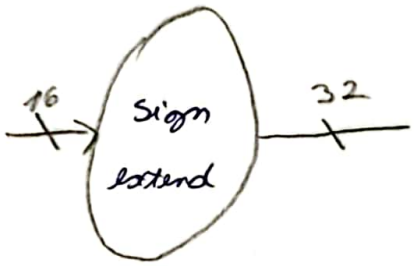
Rezultatul din UAL va fi o adresă.

Pe lângă regiștrele generale și UAL, unitățile necesare implementării acestor instrucțiuni sunt:

a) Unitatea memoriei de date



b) Unitate de extindere a semnului



Instrucțiunile sunt:

- lw $t1, valoare - deplasare ($t2)
- sw $t1, valoare - deplasare ($t2)

Aceste instrucțiuni calculează o adresă de memorie prin adunarea registrului de bază $t2 cu câmpul pentru deplasare cu semn de 16 biți din instrucțiune. Extinderea se face până la o valoare de 32 de biți cu semn.

Dacă instrucțiunea este de memorare, valoarea de memorat trebuie citită din fișierul de regiștri, unde ea se găsește în $t1.

Dacă instrucțiunea este de încărcare, valoarea citită din memorie trebuie scrisă în $t1 care este registrul specificat din fișierul de regiștri.

Scanned with CamScanner

SUBIECT: Bolio de obstă pentru instrucțiunile de încărcare sau memorare realizează un acces la un registru, format de calculare unei adrese de memorie, iar în continuare o citire sau o scriere în memorie și o scriere în fișierul de registre, dacă instrucțiunea este de încărcare.

Codul operațiilor:
-> încărcare: 35
-> memorare: 43

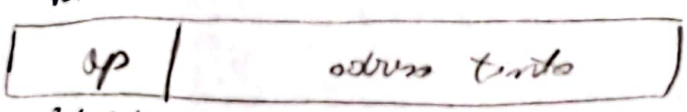
MPIS cu mai multe cicluri de ceas -> versiune abstractă.
└> diferențe față de varianta - cu un singur ciclu

Fiecare pas al execuției va necesita o perioadă de ceas => permite unei unități funcționale să fie utilizată mai mult decât o singură dată pe instrucțiune, cât timp este utilizată în cicluri diferite de ceas. Această utilizare multiplă reduce cantitatea de hardware

Avantajele MPIS - cu mai multe cicluri de ceas:
- posibilitatea de a folosi aceeași unitate funcțională de mai multe ori pe durata execuției unei singure instrucțiuni
- posibilitatea de a permite instrucțiunilor să folosească un număr de cicluri de timp.

Diferențe față de versiunea - cu un singur ciclu:
- o singură unitate de memorie, atât pentru instrucțiuni, cât și pentru date
- un singur UAL
- sunt adăugate unul sau mai multe registre oluse fiecare unități funcționale majore pentru a păstra ieșirea acelei unități până când valoarea va fi folosită într-un ciclu ulterior de ceas.

Pt J



J

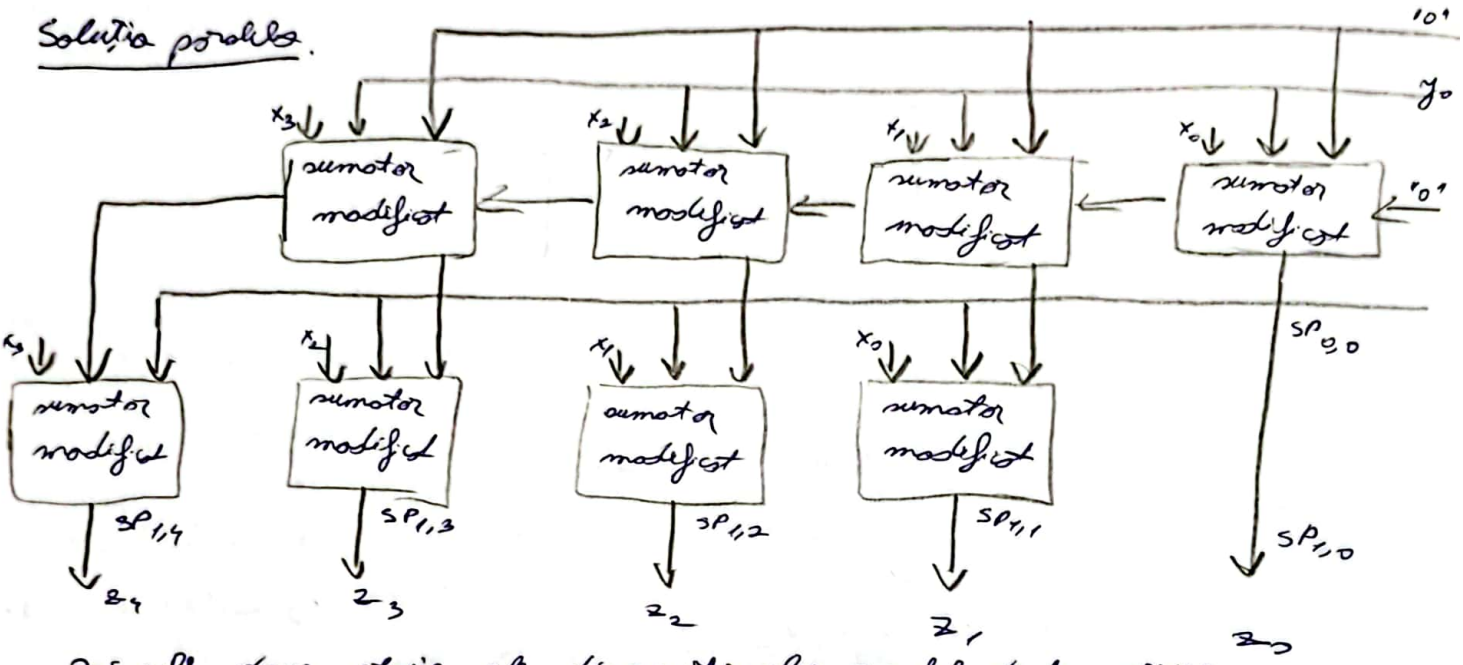
Calculează PC-ului obiectiv:
-> cei mai semnificativi 4 biți ai PC-ului curent + 4
-> câmpul imediat de adresă de 24 biți ai instrucțiunii de salt
-> cei 2 biți inferiori ai unei adrese de salt sunt întotdeauna 00.

Scanned with CamScanner

Înmulțirea

- constă în generarea și adunarea produselor parțiale obținute prin înmulțirea rangului curent al înmulțitorului cu deînmulțitul

Soluția paralelă

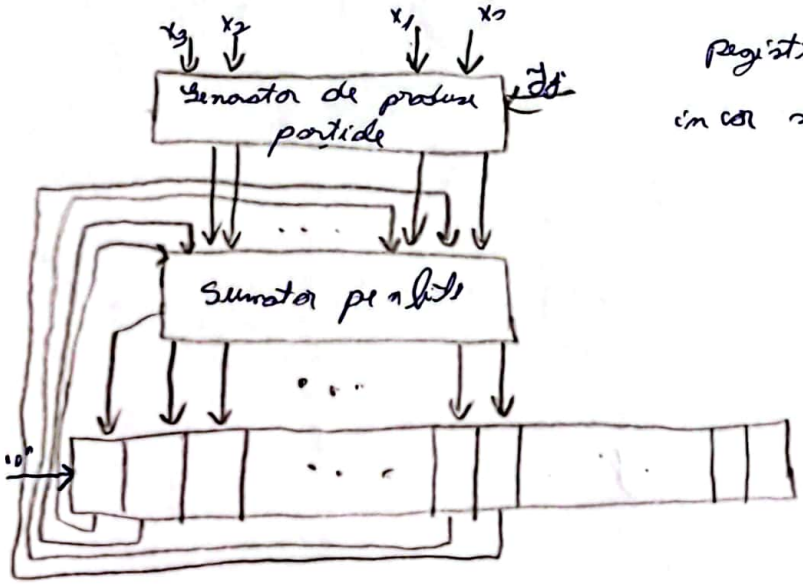


Primul două etaje ale dispozitivului paralel de înmulțire.

SP_{ij} = bitul i al sumei produselor parțiale j.

Sumatorul modificat poate fi utilizat în cadrul unei structuri de înmulțire paralelă, fiind vorba de o „programare spațială", ce conduce la viteze ridicate de operare. Soluția necesită $(n-1)^2$ sumatoare modificate.

serial-paralelă:



Registru de deplasare cu $2(n-1)$ rânguri în care se acumulează suma produselor parțiale.

Scanned with CamScanner

Alt sovis- prodil:

1. anulează conținutul registrului în care se acumulează suma produselor parțiale.
2. inițializează numărul rangului biților înmulțitorului $j = 0$.
3. formează produsul parțial $x \cdot y_j$.
4. adună produsul parțial la jumătatea superioară a registrului sumei produselor parțiale.
5. efectuează $j = j + 1$ și dacă $j = n$ treci la 8.
6. deplasează la dreapta cu un rang conținutul registrului sumei produselor parțiale;
7. treci la pasul 3.
8. produsul cu $2(n-1)$ rânguri s-a obținut în registrul de deplasare

Alg lui Booth

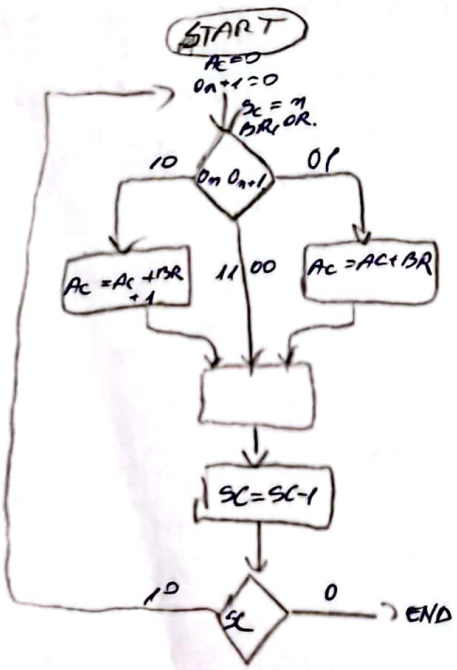
$$Y = y_{n-1} \cdot 2^{n-1} + y_{n-2} \cdot 2^{n-2} + y_{n-3} \cdot 2^{n-3} + \dots + y_1 \cdot 2^1 + y_0 \cdot 2^0 = \sum_{i=0}^{n-1} (y_{i-1} - y_i) \cdot 2^i$$

unde

- y_{n-1} reprezintă rangul de semn, codificat cu 0 în cazul numerelor pozitive / negative.
- y_{-1} constituie rangul aflat la dreapta rangului 0, având inițial val 0

$x \cdot Y = \sum_{i=0}^{n-1} x \cdot (y_{i-1} - y_i) \cdot 2^i$ → produsul parțial de rang i va fi:

y_{i-1}	y_i	produs parțial
0	0	0
0	1	$-x \cdot 2^i$
1	0	$x \cdot 2^i$
1	1	0



BR → înmulțitor.
OR → deînmulțit
AC → reg acc
E → bit transport
SC - contor de biți rang: → inițial va fi egal cu

Scanned with CamScanner

PIPELINE → etapele de execuție ale unei instrucțiuni:

1) IF - aducerea următoarei instrucțiuni aflate de PC
2) ID - decodificarea instrucțiunii
3) EX - execuția instrucțiunii
4) MEM - memorarea rezultatului și incrementarea lui PC
5) WB - scrierea în registre

Toate etapele trebuie să dureze aproximativ același timp. Un task trebuie să folosească toate etapele și ordinea să fie aceeași pentru toate task-urile.

Hazardul în pipeline

→ situația în care într-un pipe, următoarea instrucțiune nu poate fi executată în următorul ciclu de ceas

1. Hazardul structural - o instrucțiune planificată nu poate fi executată în propriul ciclu de ceas deoarece hardware-ul nu suportă combinația de instrucțiuni planificate pentru execuție.
2. Hazardul de date - instrucțiunea planificată spre execuție nu poate fi executată în ciclul de ceas urmată deoarece datele necesare execuției nu sunt încă disponibile.
3. Hazard de date de încărcare - valoarea conținută de MEM poate fi sau nu folosită de către EX. Pentru a putea determina dacă valoarea este necesară va trebui să așteptăm un ciclu de ceas adică introducem un stall. Totuși dacă folosim Forwarding, tot va trebui să stăm un ciclu de ceas.

Scanned with CamScanner

CURS - 12

- Explorarea ierarhiei de memorii.
  - Principiul localității și la baza modului de operare al programelor
    - ↳ localitatea temporală = localitatea în timp, adică se face referire la un anumit obiect și posibil ca acesta să fie referit din nou cât mai curând.
    - ↳ localitate spațială = localitate în spațiu, adică se face referire la un anumit obiect din memorie, obiectele alăturate sunt invadate cu acesta,
  - Principiul localității temporale este folosit la implementarea memoriei unui calculator sub forma unei ierarhii de memorii.
    - ↳ datele sunt copiabile între 2 nivele adiacente.
    - ↳ Bloc = unitatea minimă de informație
    - ↳ HIT = regăsirea informației necesare într-un bloc de memorie superioară (opusul = MISS)

Scanned with CamScanner

CN - EXAMEN

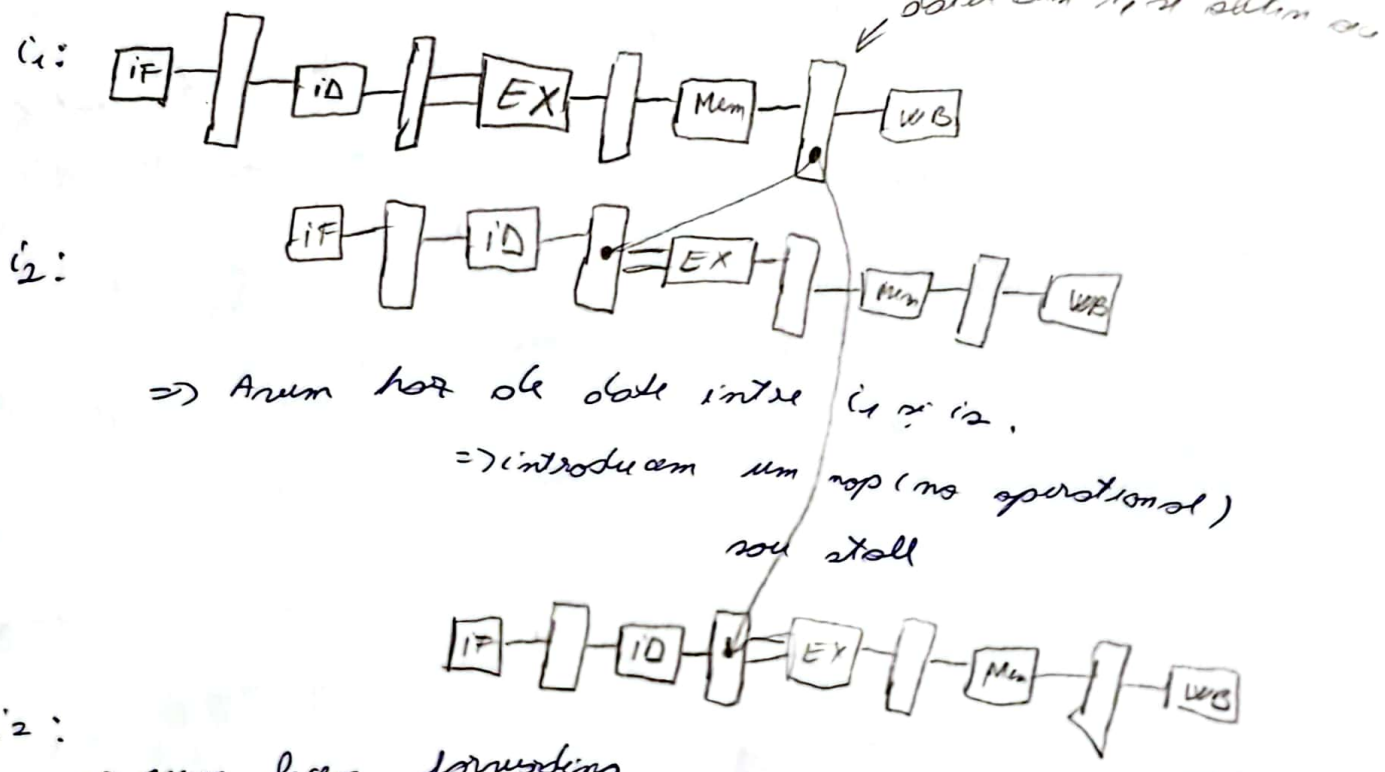
1 sub - MIPS (într-un singur ciclu de ceas sau mai multe sau microprogramat)
↳ învățat schema de implementare pt ambele tipuri de arhitecturi.

Știut formatul la R, I, J
Semnalele de semnale.

1 sub - PIPELINE
- Avem instrucțiuni. Unele pot opări hazard și cum îl corectam? + Desen

2 - sub Teorie.

Ex: i_1: lw R_1, 0(R_2) ← in reg fin de la adresa
i_2: sub R_4, R_1, R_5 ⇒ date in R_4, $R_1 - R_5$
i_3: and R_6, R_1, R_7
i_4: or R_8, R_1, R_9.



⇒ Avem hazard de date între i_1 și i_2.
⇒ introducem un nop (no operational) sau stall

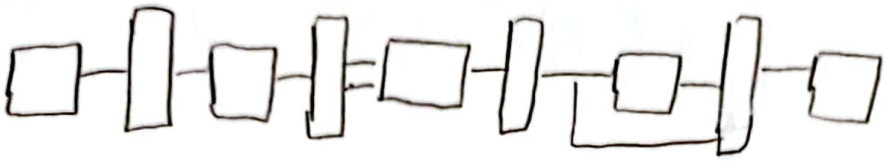
⇒ i_2:
⇒ acum facem forwarding.

Scanned with CamScanner

Pt i_3: Avem hazard și între i_1 și i_2 și între i_1 și i_3

⇒ Dacă am corectat la i_2, mai obțin încă un timp în față.

i_2:



i_3:



Pt i_3 nu trebuie nici forwarding. S-a rezolvat prin corectarea i_2.

La i_4 nu avem nicio problemă

i_4 e cu încă un timp decalat în dreapta.

2/ Alt ex:

P_{11}: lw $\$1$, 40($\$6$)

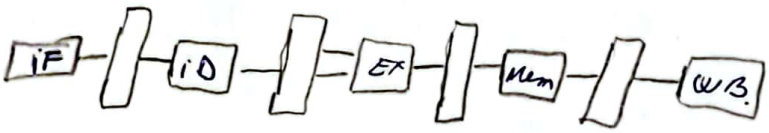
P_{12}: add $\$6$, $\$2$, $\$2$

P_{13}: sw $\$6$, 50($\$11$)

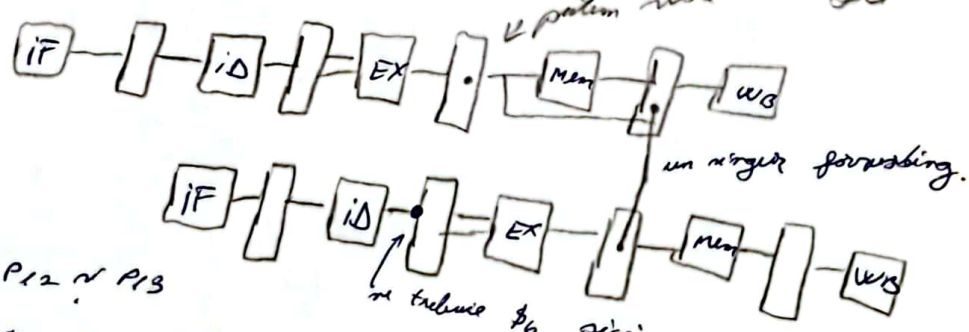
sw → extrim de la $\$6$ - datele ce aparțin la mem.

între P_{11} și P_{12} nu avem hazard.

P_{11}: lw



P_{12}: add



⇒ Este un hazard între P_{12} și P_{13}

Trebuie un fel între P_{11} și P_{13}

Scanned with CamScanner

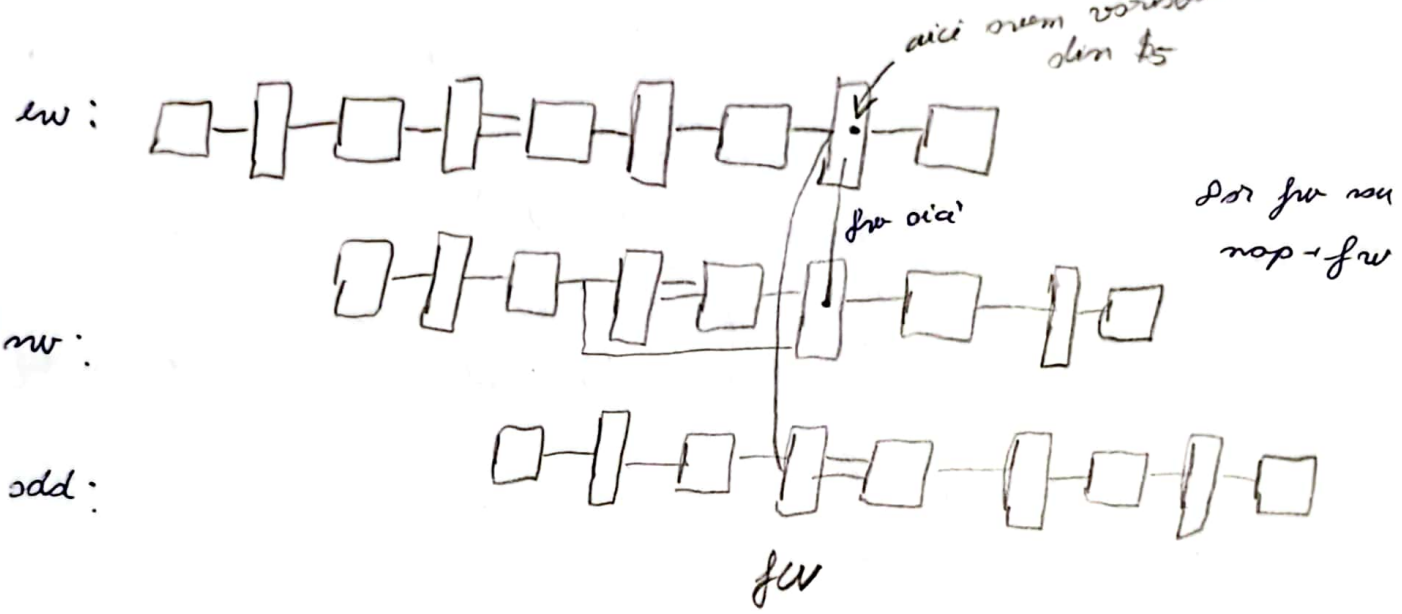
3/

P_{21}: lw : $5, -16($5)

P_{22} : sw $5, -16($5)

P_{23} : add $5, $5, $5

sw: se scrie în mem



Ex: Se consideră următ de cod:

lw: $1, 40($6)

add: $5, $5, $5

a) În cadrul execuției ac. instr, ce se memorează în fiecare registru de pipe?

b) Cum reg memorează masca citirii și de către ce sunt citite fiecare?

c) EX → calculăm adresa

MEM → extragem datele de la adr calculată anterior. (la lw)

(la ADD): EX → introducem datele în ADD

MEM → nimic.

Scanned with CamScanner